

Linear programming



Back in the USSR

- Let's say you're running a shoe factory.
- There are three types of shoe you can make: dress shoes, Wellington boots and high heels.
- Dress shoes require 3 units of leather and no plastic, Wellingtons require 3 units of plastic and no leather, and high heels require 1 unit of leather and 1 unit of plastic.
- Comrade Brezhnev has sent you 1000 units of leather and 1000 units of plastic.
- What should you do?

Back in the USSR

- Write x_1, x_2 and x_3 for the number of dress shoes, Wellingtons and high heels respectively.
- (Remember dress: 3 leather, wellie: 3 plastic, high heel: 1 leather + 1 plastic)
- You want to maximise $x_1 + x_2 + x_3$.
- But there are constraints.
- You only have 1000 leather, so $3x_1 + x_3 \leq 1000$
- You only have 1000 plastic, so $3x_2 + x_3 \leq 1000$
- And of course $x_1, x_2, x_3 \geq 0$

Linear programming

- A linear program in *standard form* is:
- maximise $\sum_{i \in V} c_i x_i$
- subject to
 - $\sum_{j \in V} a_{ij} x_j \leq b_i$ for each $i \in C$
 - $x_i \geq 0$ for each $i \in V$.
- Note any linear program can be put in this form (e.g. if we had a problem to minimise $\sum_i d_i x_i$ then we can equivalently say maximise $\sum_i (-d_i) x_i$)

Linear programming

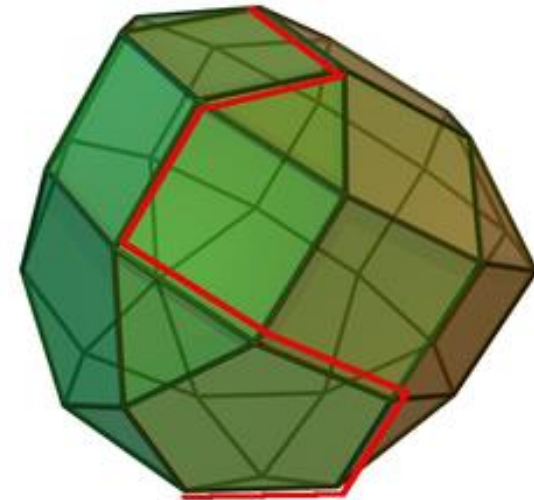
- Exercise: maybe we think Wellington boots are twice as good as the other kinds. How should we change the problem to take this into account?
- Answer: change the objective to $x_1 + 2x_2 + x_3$.

Exercises

- GT R-26.1, R-26.3, R-26.7, A-26.3

The Simplex Algorithm

- Main take-home message: **linear programs can be solved efficiently**
- The classic algorithm is the ‘simplex algorithm’
- Basically this runs along the edges of the feasible region, trying to increase the objective function
- We won’t go into the details, but see Section 26.2 of the textbook
- Usually works well in practice, but can be exponential in the worst case
- There are poly-time ‘interior point’ methods



Back to the shoe factory

- maximise $x_1 + x_2 + x_3$
- subject to $3x_1 + x_3 \leq 1000$, $3x_2 + x_3 \leq 1000$, $x_1, x_2, x_3 \geq 0$
- I claim the best thing to do is just make 1000 high heels (this may tell you something about why the Soviet Union failed...)
- But how can I convince you of this?
- I want to show $x_1 + x_2 + x_3 \leq 1000$
- If I add together my two constraints, I get $3x_1 + 3x_2 + 2x_3 \leq 2000$
- But $3x_1 + 3x_2 + 2x_3 \geq 2x_1 + 2x_2 + 2x_3$
- So $2x_1 + 2x_2 + 2x_3 \leq 2000$, so $x_1 + x_2 + x_3 \leq 1000$ 😊

LP duality

- We can do this in general. Remember our problem is
- maximise $\sum_{i \in V} c_i x_i$
- subject to
 - $\sum_{j \in V} a_{ij} x_j \leq b_i$ for each $i \in C$
 - $x_i \geq 0$ for each $i \in V$.
- Let's say we add y_i times constraint i
- Then what I get is $\sum_i y_i \sum_j a_{ij} x_j \leq \sum_i y_i b_i$

LP duality

- Then what I get is $\sum_i y_i \sum_j a_{ij} x_j \leq \sum_i y_i b_i$
- I want this to give me a bound on $\sum_j c_j x_j$
- So what I need is that the coefficient of each x_j is at least c_j
- So I need $\sum_i a_{ij} y_i \geq c_j$ for each $j \in V$.
- And I want to get as good a bound as possible, so I want to minimise $\sum_i y_i b_i$
- And to be able to add the inequalities, I need each $y_i \geq 0$
- This looks like another linear program! (Not in standard form)

LP duality

- ‘Primal’: maximise $\sum_{i \in V} c_i x_i$
- subject to
 - $\sum_{j \in V} a_{ij} x_j \leq b_i$ for each $i \in C$
 - $x_i \geq 0$ for each $i \in V$.

Maximise $\mathbf{c} \cdot \mathbf{x}$
Subject to $\mathbf{Ax} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}$

- ‘Dual’: minimise $\sum_{i \in C} b_i y_i$
- subject to
 - $\sum_{i \in C} a_{ij} y_i \geq c_j$ for each $j \in V$
 - $y_i \geq 0$ for each $i \in C$

Minimise $\mathbf{b} \cdot \mathbf{y}$
Subject to $\mathbf{y}^t \mathbf{A} \geq \mathbf{c}, \mathbf{y} \geq \mathbf{0}$

LP duality: shoes

- Primal:
- maximise $x_1 + x_2 + x_3$
- subject to $3x_1 + x_3 \leq 1000$
 $3x_2 + x_3 \leq 1000$
 $x_1, x_2, x_3 \geq 0$

Optimum: 1000 (0,0,1000)

- Dual:
- minimise $1000y_1 + 1000y_2$
- subject to $3y_1 \geq 1$
 $3y_2 \geq 1$
 $y_1 + y_2 \geq 1$
 $y_1, y_2 \geq 0$

Optimum: 1000 (0.5,0.5) – or (1/3,2/3), or (2/3,1/3),
or anything in between

LP Duality

Theorem ('Strong LP duality'): The primal and dual problems have the same optimum value.

Shadow prices

- It turns out we can interpret the y_i as a kind of 'price' information
- Imagine constraint i corresponds to a raw material (leather/plastic)
- We could buy more and produce more, or sell some and produce less
- A dual solution y_i is a set of prices at which we are indifferent about doing this ('equilibrium')
- e.g. if leather and plastic cost €0.50 each, then buying an extra unit of each and making an extra shoe breaks even
- These are called 'shadow prices'
- Those damn capitalists get everywhere!

Exercises

- GT R-26.12, R-26.13
- CLRS 29.1-4, 29.2-1

LP is all you need

- These linear programs seem quite simplistic, but actually they can express lots of stuff.
- Exercise: express Vertex-Cover as an LP problem
- Hint: have a variable x_v for each vertex representing whether we include that vertex in our cover

Vertex-Cover

- Minimise $\sum_{v \in V} x_v$
- Subject to $x_v + x_w \geq 1$ for each $vw \in E$
 $0 \leq x_v \leq 1$ for each $v \in V$
- Or in standard form
- Maximise $\sum_{v \in V} -x_v$
- Subject to $-x_v - x_w \leq -1$ for each $vw \in E$
 $x_v \leq 1$ for each $v \in V$
 $x_v \geq 0$ for each $v \in V$

Wait a minute

- I told you earlier that LP problems can be solved in polynomial time
- And Vertex-Cover is NP-complete
- So have I just proved that $P=NP$?

The importance of being integral

- No! There's one condition we forgot
- Suppose our graph is a triangle
- Then the LP is minimise $x_1 + x_2 + x_3$
 subject to $x_1 + x_2 \geq 1$
 $x_2 + x_3 \geq 1$
 $x_3 + x_1 \geq 1$
 $x_1, x_2, x_3 \geq 0$
- The optimum value is 1.5 (0.5,0.5,0.5).
- What vertex cover does that correspond to?
- The x_i also have to be integers!

- Maximise $\sum_{v \in V} -x_v$
- Subject to $-x_v - x_w \leq -1$ for each $vw \in E$
 - $x_v \leq 1$ for each $v \in V$
 - $x_v \geq 0$ for each $v \in V$
 - $x_v \in \mathbb{Z}$ for each $v \in V$

The importance of being integral

- It turns out requiring variables to be integers makes all the difference
- None of the efficient algorithms work 😞
- (Actually this kind of thing is quite common: integers make things hard)
- **Integer** Linear Programming (ILP) is NP-complete
- ...as we just saw by reduction from Vertex-Cover
- But we can still get something from the LP formulation

Bounds

- Remember for branch-and-bound I wanted to be able to get a **bound** on how good a solution could be
- Suppose I have a hard problem formulated as an ILP (maximization) problem: how can I get a bound on how good any solution can be?
- Just forget the integrality constraints! (**'Fractional relaxation'**)
- Removing constraints can only increase the maximum achievable value
- So I know $m_{\text{true}} = m_{\text{ILP}} \leq m_{\text{LP}}$ (or the other way around for a minimisation)
- For example the LP told me that a triangle doesn't have a vertex cover with less than 1.5 vertices
- In fact the true minimum is 2

Branch-and-bound

- So to get a bounding method to plug into branch-and-bound:
 - Given a partial assignment, set the assigned variables to their assigned values, and solve the fractional relaxation of the remaining problem (i.e. without integrality constraints)
 - The optimum value is a bound
-
- We can also use the LP to help us decide how to branch
 - If the fractional LP makes a variable far from an integer that might be a good variable to branch
 - Also *rounding* the LP solutions may give us good solutions to compare the bounds to

Recipe

- So a recipe for solving problems in NP is:
 - Express the problem as an ILP (possible since ILP is NP-complete)
 - Do branch-and-bound on the ILP, using the fractional relaxation to give bounds
 - Profit
- Or if you're feeling lazy just use an ILP solver written by someone else
- **Important note:** this is not guaranteed to run in polynomial time, or any reasonable amount of time (so if you can think of a good algorithm that's better!)
 - But it is often quite good in practice

3-SAT as an ILP

- Let's see how to express 3-SAT as an ILP
- Of course, we know 3-SAT can be reduced to Vertex-Cover and we saw how to express Vertex-Cover as an ILP
- So we could take our 3-SAT instance, convert it to a Vertex-Cover instance and then express that as an ILP
- But usually it's better to go direct

3-SAT as an ILP

- Maximise 1 subject to

$$x_i + x_j + x_k \geq 1 \text{ for clauses } x_i + x_j + x_k$$

$$x_i + x_j + (1-x_k) \geq 1 \text{ for clauses } x_i + x_j + \neg x_k \text{ a.k.a. } x_i + x_j - x_k \geq 0$$

$$x_i + (1-x_j) + (1-x_k) \geq 1 \text{ for clauses } x_i + \neg x_j + \neg x_k \text{ a.k.a. } x_i - x_j - x_k \geq -1$$

$$(1-x_i) + (1-x_j) + (1-x_k) \geq 1 \text{ for clauses } \neg x_i + \neg x_j + \neg x_k \text{ a.k.a. } -x_i - x_j -$$

$$x_k \geq -2$$

$$x_i \leq 1 \text{ for each } i$$

$$x_1, \dots, x_n \geq 0, x_1, \dots, x_n \in \mathbb{Z}$$

3-SAT as an ILP

- This illustrates an important point: an (I)LP can just have no solution
- e.g. maximise x_1 subject to
$$\begin{aligned}x_1 &\leq 1 \\ -x_1 &\leq -2\end{aligned}$$
- In this case we say the optimum value is “ $-\infty$ ” (and similarly for a minimisation problem $+\infty$)

Clique number

CLIQUE

Input: a graph G and an integer k

Question: does G contain a k -clique? (k vertices with every pair joined by an edge)

We will consider the optimisation version of this, i.e. given a graph G find the maximum k such that G contains a k -clique.

Exercise: express clique number as an ILP. Hint: use variables x_v for $v \in V$, representing whether v is in the clique.

Clique number

- Maximise $\sum_{v \in V} x_v$
- Subject to $x_v + x_w \leq 1$ for each $vw \notin E$ (i.e. for each v and w that are not joined by an edge)
 - $x_v \leq 1$ for each $v \in V$
 - $x_v \geq 0, x_v \in \mathbb{Z}$ for each $v \in V$

Exercises

- GT C-26.10, C-26.13, A-26.5, A-26.6
- CLRS 29.2-6, 29.2-7